

T07 · 플랜두씨 다이어리 2

가입하고, 로그인하고, 내 것만 본다

과제 6에 로그인·소유권 기반 RLS를 붙여 계정마다 자기 자료만 보이게 만든 인증 구현 설명서 (7.md)

결과물 주소	https://aleph-pds-auth.vercel.app
소스 주소	https://github.com/ginye28/ALEPH/tree/main/plandosee-auth
과제 6 결과물	https://aleph-pds.vercel.app (별개 프로젝트, 건드리지 않음)
백엔드	Supabase Postgres, 새 프로젝트(plandosee-auth) — 과제 6과 별개
촬영	2026년 9월 8일 16:26:28 · https://aleph-pds-auth.vercel.app
검사	PASS 40 / FAIL 0 (총 40개)

1. 검증 안내서

① 어디로 가나요

`https://aleph-pds-auth.vercel.app` — 로그인 화면부터 시작

② 세 단계 안에 무엇을 하나요

1. 새 계정으로 **가입**합니다.
2. **로그인**해 계획 하나를 만듭니다.
3. **로그아웃**한 뒤 같은 자료 화면 주소를 다시 엽니다.

③ 무엇이 보이면 통과인가요

- 가입 직후 자동으로 로그인되거나 로그인 화면으로 이동합니다.
- 만든 계획이 **내 화면에만** 보입니다.
- 로그아웃 뒤에는 로그인 화면만 보이고 자료는 보이지 않습니다.

④ 안 될 때는 무엇이 보이나요

- 가입이 안 되면 이미 있는 이메일인지 오류 문구를 읽습니다.
- 로그인 후에도 로그인 화면이면 새로고침 한 번 해 봅니다(세션 반영 지연).
- 자료가 안 보이면 계정을 잘못 골랐는지(다른 계정으로 가입했는지) 확인합니다.

① 무엇으로 붙였나

직접 구현·라이브러리·인증 서비스 셋 중 **인증 서비스**를 골랐습니다. **Supabase Auth(GoTrue)**, **이메일+비밀번호** 방식 이고, 브라우저에서는 `@supabase/supabase-js` 로 가입·로그인·로그아웃·세션 유지를 전부 처리합니다. 이미 과제 6에서 Supabase Postgres + RLS를 쓰고 있어, 인증과 데이터 접근 제어가 같은 플랫폼의 `auth.uid()` 로 묶입니다.

항목	내용
고른 갈래	인증 서비스 (직접 구현 아님)
서비스	Supabase Auth (GoTrue) — 이메일+비밀번호
클라이언트 라이브러리와 버전	<code>@supabase/supabase-js</code> 2.112.4 (package.json 범위 <code>^2.58.0</code> , package-lock.json에 잠긴 실제 버전)

로그인

플랜두씨 다이어리 2 — 로그인해야 내 기록을 볼 수 있습니다

로그인하지 않으면 자료 화면 자체가 열리지 않습니다. 계정마다 자기 계획·할일·실행기록만 보입니다.

이메일

me@example.com

비밀번호

6자 이상

로그인

처음이라면 가입하기

01_비로그인_로그인화면 — 로그인 폼만 있고 자료 화면 요소는 없음 — 로그인하지 않으면 자료 화면 자체가 열리지 않는다는 안내가 첫 화면에 그대로 있음(검사 15:21)

가입하기

플랜두씨 다이어리 2 — 로그인해야 내 기록을 볼 수 있습니다

로그인까지 않으면 자료 화면 자체가 열리지 않습니다. 계정마다 자기 계획·할일·실행기록만 보입니다.

이메일

pds-auth-shot-a-1788852367767@example.com

비밀번호

가입하기이미 계정이 있습니다

02_가입_입력

플랜두씨 다이어리 2

계획(Plan) → 할 일(Do) → 돌아보기(See) · 기준 시간대 Asia/Seoul (UTC+9) 오늘 2026-09-08 pds-auth-shot-a-1788852367767@example.com

계획

계획은 기록도 지워지지 않고, 세 계정으로만 열립니다

제목	기간	우선순위	해상 시간	계정
----	----	------	-------	----

새 계획

기준 시간대 Asia/Seoul (UTC+9)

제목

예: ALEPH 과제 6 진행

시작일

연도-월-일

종료일

연도-월-일

우선순위

보통

해상 시간 (분)

600

생성 기준

무엇이 되면 이 계획을 완료로 볼지 한두 문장으로

메모 (선택)

계획 만들기

할 일

먼저 위에서 계획을 하나 선택하거나 만듭니다.

실행 기록

할 일 목록에서 실행기록을 선택하세요

위 할 일 목록에서 "실행기록"을 눌러 어느 할 일에 적을지 고릅니다.

돌아보기

먼저 계획을 하나 선택합니다.

날짜별 실제 시간

그날 실행기록의 실제 걸린 시간 합계 · 단위 분 · Asia/Seoul 기준

아직 실행기록이 없습니다. 할 일을 고르고 실행기록을 남기면 여기에 날짜별로 보입니다.

내 자료 내보내기

계획·이력·할일·실행기록·고칠 할 전체를 내보낼 때

전체 내보내기

내 계정

pds-auth-shot-a-1788852367767@example.com

⚠ 위험 구역

계정 삭제는 내 계획·할일·실행기록·고칠 것을 전부 지웁니다. 다만 가입 정보(이메일) 자체는 이 버전으로 지워지지 않고 별도 절차가 필요합니다 — Auth 계정 레코드의 하드 삭제는 관리자 권한이 있어야 해 이 화면에서는 못 만듭니다.

내 계정 삭제

계획·할일·실행기록은 서버의 실제 데이터베이스에 저장되고, 로그인한 내 계정 소유의 행만 서버(RLS)가 돌려줍니다.

03_가입_직후_메인화면

계정 A(pds-auth-shot-a-1788852367767@example.com) 가입 직후 자동 로그인, 자료 화면(계획 없음) 표시 — 가입 → 로그인인 이 어짐(검사 18)

로그인

플랜두씨 다이어리 2 — 로그인해야 내 기록을 볼 수 있습니다

로그인하지 않으면 자료 화면 자체가 열리지 않습니다. 계정마다 자기 계획·할일·실행기록만 보입니다.

이메일

비밀번호

로그인

처음이라면 가입하기

과제 6에 넣어 둔 내 자료가 이 계정으로 옮겨졌습니다 (T07-C100)

"옮겼습니다"라는 문장이나 이전 코드가 있다는 사실은 근거가 되지 않습니다. 과제 6에서 내보낸 파일과 과제 7 계정에서 내보낸 파일을 **행 id로 맞대어** 셧습니다 — id가 그대로라는 것은 다시 입력한 것이 아니라 실제로 옮겨졌다는 뜻입니다.

PASS (검사 39) 판정 근거: 과제 6 내보내기(plandosee-내보내기-2026-09-02-과제7이전용.json)의 27건이 과제 7 계정 내보내기(plandosee-auth-내보내기-2026-09-08.json)에 id 그대로 전부 있음 — plans 4건 중 4건 · planRevisions 5건 중 5건 · tasks 7건 중 7건 · executionRecords 5건 중 5건 · reviewNotes 6건 중 6건 · 빠진 행 0건 (id가 같다는 것은 다시 입력한 것이 아니라 실제로 옮겨졌다는 뜻입니다)

계정이 있는지 없는지를 흘리지 않습니다 (T07-C99)

이메일 또는 비밀번호가 올바르지 않습니다.

05_오류_비밀번호틀림 — 있는 계정에 비밀번호만 틀린 경우

이메일 또는 비밀번호가 올바르지 않습니다.

06_오류_계정없음 — 아예 없는 계정인 경우

두 화면의 문구가 글자까지 같습니다 — 비밀번호틀림 "이메일 또는 비밀번호가 올바르지 않습니다." / 계정없음 "이메일 또는 비밀번호가 올바르지 않습니다." — 그래서 오류 문구만 보고 "이 이메일은 가입돼 있다"를 알아낼 수 없습니다. **PASS (검사 20)**

같은 이메일로 두 번 가입되지 않습니다 (T07-C98)

이미 가입된 이메일입니다.

07_중복가입_거절 — 이미 있는 이메일로 다시 가입 시도 → 거절 문구 — 같은 이메일 재가입이 막힘(검사 19) **PASS (검사 19)**

② 왜 그걸 골랐나

검토한 방법	고르지 않은 이유
직접 구현 (bcrypt + 세션 쿠키를 손으로 짤)	비밀번호 저장·세션 발급·만료·재설정을 전부 직접 검증해야 해서 실수 여지가 크다. 이 과제의 배점은 '많이 쓰이는 것을 잘 골랐는지'이지 인증을 재발명했는지가 아니다.
Auth.js (NextAuth)	서버 세션·미들웨어 전제가 강해 지금의 Vite SPA + 정적 배포 구조와 맞지 않는다. 과제 6부터 지켜온 '서버 없이 브라우저에서만' 원칙이 깨진다.

③ 어디를 어떻게 고쳤나

흐름	소스 위치
가입·로그인·로그아웃·세션 조회	<code>src/api/auth.js</code> (signUp/signIn/signOut/getSession/onAuthStateChange)
세션 없으면 자료 화면 자체를 렌더하지 않음	<code>src/components/AuthGate/AuthGate.jsx</code>
가입/로그인 폼, 동일 오류 문구 처리	<code>src/components/AuthForm/AuthForm.jsx</code>
로그아웃·계정 삭제(내 데이터 행)	<code>src/components/AccountSection/AccountSection.jsx</code> , <code>src/api/auth.js#deleteMyData</code>
소유자 칼럼 강제 스탬프 + 소유자 기반 RLS	<code>supabase/schema.sql</code> (stamp_owner() 트리거 + auth.uid()=user_id 정책, 표 5개 전부)
과제 6 실 데이터 이전	<code>src/api/migrateFromT06.js</code>

④ 안 열리는 것을 확인한 기록

확인 다섯 가지입니다. 각 줄의 왼쪽이 **제대로 된 요청이 성공한 장면**, 오른쪽이 **같은 자리에서 잘못된 요청이 거절된 장면**입니다. 아래 카드 2·3·4가 각각의 자세한 기록입니다.

확인	성공한 요청	거절된 요청
1 · 로그인 없이 자료를 열기	로그인한 계정에서는 자료 화면이 열리고 내 계획이 보임 (08_계정A_자료 화면)	화면에서는 로그인 폼만 있고 자료 화면 요소가 DOM에 아예 없음 PASS (검사 21) 화면을 건너뛰고 REST 주소를 직접 두드려도 401 로 막힘 PASS (검사 32)
2 · 저장된 비밀번호 읽기	정상 비밀번호로 로그인 성공 (검사 18)	<code>auth.users.encrypted_password</code> 는 <code>\$2a\$10\$...</code> 해시뿐 — 입력한 글자가 보이지 않고, 같은 비밀번호로 만든 두 계정의 값도 서로 다름 PASS (검사 22)
3 · 로그아웃 뒤 같은 토큰 재사용	로그인 상태의 같은 요청은 200으로 내 계획 2건 반환	로그아웃 뒤 같은 주소·같은 방식·같은 토큰 으로 다시 요청 — 달라진 것은 로그아웃 여부뿐인데 자료 0건 PASS (검사 23)
4 · 남의 자료 읽기·수정·삭제 (양방향)	각 계정은 자기 계획을 id로 읽고 고치고 지울 수 있음 (검사 1·2·17)	A↔B 양방향 모두 읽기 거절(null)·삭제 미반영(deleted_at 그대로 null) PASS (검사 26) PASS (검사 27)
5 · 남의 계정을 적어 보내기 / 목록 섞임	내 목록에는 내 계획이 전부 나옴 (A 2건 · B 1건)	요청 본문에 A의 user_id를 적어 보내도 저장된 행은 로그인한 B로 찍히고, 목록 응답에 상대 계정 행 0건 PASS (검사 29) PASS (검사 28)

카드 2 — 비밀번호를 어떻게 맡아 두는지 보이기

GoTrue는 비밀번호를 **bcrypt**로 해시해 `auth.users.encrypted_password` 에 저장합니다. 이 코드베이스 어디에도 비밀번호를 직접 저장·비교하는 로직이 없습니다 — 해시 함수를 손으로 부르는 자리가 아예 없다는 것이 이 카드의 답입니다.

bcrypt로 남겨 둔 이유 — argon2가 더 최근 방식이지만, 그것을 쓰려면 인증을 직접 구현하는 쪽으로 돌아가야 합니다. bcrypt는 계정마다 무작위 salt를 자동으로 붙이고 반복 횟수(cost)를 두어 대입 속도를 늦추는, 오래 검증된 방식입니다. 검증된 기본값을 그대로 쓰는 편이 방식을 한 단계 올리려다 직접 구현으로 되돌아가는 것보다 안전하다고 보고 GoTrue의 bcrypt(cost 10)를 그대로 받아들였습니다. 아래는 실제로 그렇게 저장돼 있는지 눈으로 확인한 기록입니다.

실제로 확인한 해시값 (2026-09-02, Supabase SQL 편집기)

같은 비밀번호로 만든 두 스크래치 계정의 `encrypted_password` 를 직접 조회했습니다:

계정	user id	encrypted_password
A	3f36b1c9-58f9-4eb9-b31b-a8b0d7566cfc	\$2a\$10\$/Rpbk76a5Spss.LKsR/PnO48cRQhqQ3NkkmSEY6N/iM/IQX6cHGBm
B	0ef5b2d6-6cf4-4064-ac06-0998aedef5a9b	\$2a\$10\$TXosJlzyWWWoRQwW8lw9uci/oJh/FV4EUGRyZZ2Np8sM2IdeTcMa

둘 다 `$2a$10$...` (bcrypt, cost 10) 포맷이고, **같은 비밀번호인데도 해시가 완전히 다릅니다** — 계정마다 무작위 salt가 자동으로 붙었다는 뜻입니다. **PASS (검사 22)**

비밀번호 원문이 어디에 남는지 네 곳을 열어 봤습니다 (T07-C105, T07-C106)

로그인은 비밀번호를 서버로 보내야 성립합니다 — 관건은 그 뒤입니다. 실제 로그인 요청 `POST /auth/v1/token?grant_type=password` 를 직접 보내고, 원문이 남았을 만한 네 곳을 문자열로 뒤졌습니다.

열어 본 곳	결과
로그인 응답 본문	원문 0건 — 돌아온 것은 <code>access_token·token_type·expires_in·expires_at·refresh_token·user·weak_password</code> 뿐
<code>localStorage</code> 에 저장된 세션	원문 0건 — 토큰만 들어 있음
화면에 그려진 글자(<code>document.body.innerText</code>)	원문 0건
주소창(<code>location.href</code>)	원문 0건

PASS (검사 37) 판정 근거: `POST /auth/v1/token?grant_type=password → 200` · 응답 본문에 비밀번호 원문 0건 (돌아온 것은 `access_token·token_type·expires_in·expires_at·refresh_token·user·weak_password`) · `localStorage`에 저장된 세션·화면 글자·주소창까지 네 곳 모두 원문 0건

카드 3 — 들어온 사람을 어떻게 기억하는지 보이기

로그인 성공 시 **JWT 액세스 토큰**(만료 1시간) + **리프레시 토큰**을 받아 브라우저 `localStorage`에 저장합니다. 이메일 + 비밀번호 흐름은 리다이렉트가 없어 URL 쿼리에 토큰이 실리지 않습니다.

확인	결과
액세스 토큰이 URL에 있는가	access_token/refresh_token 문자열이 주소 어디에도 없음(검사 24) — 스크린샷 대신 주소 문자열 그대로 기록 — <code>https://aleph-pds-auth.vercel.app/</code> PASS (검사 24)
토큰 만료 시각	발급 후 정확히 3600초(1시간) PASS (검사 25)
로그인 상태 조회 → 로그아웃 뒤 같은 토큰 재사용	자료 0건으로 거절됨 (검사 23) — 로그인 상태 조회 200(내 계획 2건) → 로그아웃 뒤 같은 토큰 재사용 200이지만 0건 — <code>session_is_active()</code> 가 <code>auth.sessions</code> 에서 지워진 세션을 찾지 못해 행을 전부 걸러냄(토큰이 서명상 유효해도 자료는 하나도 못 봄)

로그아웃 즉시 세션을 무효화합니다. JWT 액세스 토큰 자체는 stateless라 `signOut()` 만으로는 서명이 만료 전까지 계속 "유효"합니다. 그래서 `auth.uid()` 검사만으로는 부족합니다 — 대신 로그인마다 `auth.sessions`에 생기는 세션 행을 `signOut()`이 지운다는 점을 이용해, 모든 RLS 정책에 토큰의 `session_id` 클레임이 **지금도 `auth.sessions`에 살아 있는지**를 함께 검사하는 `session_is_active()` 함수를 추가했습니다(`supabase/schema.sql`). 로그아웃한 순간 그 행이 지워지므로, 훔친 토큰을 계속 들고 있어도 상태 코드는 200이지만 내 자료는 단 한 줄도 돌아오지 않습니다.

카드 4 — 남의 자료가 안 열리는 것을 보이기

모든 표에 `user_id` 를 직접 두고, `stamp_owner()` 트리거가 INSERT마다 클라이언트가 보낸 값과 무관하게 항상 `auth.uid()` 로 덮어씁니다. RLS는 `auth.uid() = user_id` 일 때만 select/insert/update를 허용합니다.

플랜두씨 다이어리 2

계획[Plan] → 실제로 한 일[Do] → 돌아보기[See] · 기준 시간대 Asia/Seoul (UTC+9)

오늘 2026-09-08

pds-auth-shot-a-1788852367767@example.com

계획

계획은 고쳐도 지워지지 않고, 세 개정본으로 합입니다

제목

기간

우선순위

대상 시간

계정

[참시] A의 계획

2026-09-01 ~ 2026-09-05

보통

60분

1판

선택됨

이력 보기

고지거

삭제

새 계획

기준 시간대 Asia/Seoul (UTC+9)

제목

예: ALEPH 과제 6 진행

시작일

연도-월-일

종료일

연도-월-일

우선순위

보통

대상 시간 (분)

600

생성 기준

무엇이 되면 이 계획을 완료로 볼지 한두 문장으로

메모 (선택)

계획 만들기

할 일

장황: 짧은 순서 · 내림차순 · 같이 같으면 id 순

검색 (제목)

상태

제목으로 찾기

전체

우선순위

전체

할일 기준

만든 순서

방향

내림차순

제목

대상일

우선순위

태그

대상 시간

방향

조각에 맞는 할 일이 없습니다.

제목

예: 스카마 SQL 작성

대상일 (선택)

연도-월-일

우선순위

보통

대상 시간 (분)

90

태그 (필요로 구분, 선택)

백엔드, 급함

설명 (선택)

할 일 추가

실행 기록

할 일 목록에서 실행기록을 선택하세요

위 할 일 목록에서 "실행기록"을 눌러 어느 할 일에 적용지 고릅니다.

돌아보기

[참시] A의 계획

0

전체 할 일

0

완료

0

지연

0

미함

대상 시간 합계

0분

실제 시간 합계

0분

차이 (실제 - 예상)

0분

고질 점 한 줄

예: 마감일을 너무 낙관적으로 잡았다 — 다음엔 1.5배로

고질 점 저장

날짜별 실제 시간

그날 실행기록의 실제 걸린 시간 합계 · 단위 분 · Asia/Seoul 기준

아직 실행기록이 없습니다. 할 일을 고르고 실행기록을 남기면 여기에 날짜별로 합입니다.

내 자료 내보내기

계획 · 이력 · 할일 · 실행기록 · 고질 점 전체를 파일 하나로

전체 내보내기

내 계정

pds-auth-shot-a-1788852367767@example.com

로그아웃

△ 위험 구역

계정 삭제는 내 계획 · 할일 · 실행기록 · 고질 점을 전부 지웁니다. 다만 가입 정보(이메일) 자체는 이 버전으로 지워지지 않고 별도 절차가 필요합니다 — Auth 계정 레코드의 하드 삭제는 관리자 권한이 있어야 해 이 화면에서는 못 만들었습니다.

내 계정 삭제

계획 · 할일 · 실행기록은 서버의 실제 데이터베이스에 저장되고, 로그인한 내 계정 소유의 행만 서버[Redis]가 돌려줍니다.

08_계정A_자료화면

플랜두씨 다이어리 2

계획[Plan] → 실제로 한 일[Do] → 돌아보기[See] · 기준 시간대 Asia/Seoul (UTC+9)

오늘 2026-09-08

pds-auth-shot-b-1788852367767@example.com

계획

계획은 고쳐도 지워지지 않고, 세 개정본으로 합입니다

제목

기간

우선순위

대상 시간

계정

새 계획

기준 시간대 Asia/Seoul (UTC+9)

제목

예: ALEPH 과제 6 진행

시작일

연도-월-일

종료일

연도-월-일

우선순위

보통

대상 시간 (분)

600

생성 기준

무엇이 되면 이 계획을 완료로 볼지 한두 문장으로

메모 (선택)

계획 만들기

할 일

먼저 위에서 계획을 하나 선택하거나 만듭니다.

실행 기록

할 일 목록에서 실행기록을 선택하세요

위 할 일 목록에서 "실행기록"을 눌러 어느 할 일에 적용지 고릅니다.

돌아보기

먼저 계획을 하나 선택합니다.

날짜별 실제 시간

그날 실행기록의 실제 걸린 시간 합계 · 단위 분 · Asia/Seoul 기준

아직 실행기록이 없습니다. 할 일을 고르고 실행기록을 남기면 여기에 날짜별로 합입니다.

내 자료 내보내기

계획 · 이력 · 할일 · 실행기록 · 고질 점 전체를 파일 하나로

전체 내보내기

내 계정

pds-auth-shot-b-1788852367767@example.com

로그아웃

△ 위험 구역

계정 삭제는 내 계획 · 할일 · 실행기록 · 고질 점을 전부 지웁니다. 다만 가입 정보(이메일) 자체는 이 버전으로 지워지지 않고 별도 절차가 필요합니다 — Auth 계정 레코드의 하드 삭제는 관리자 권한이 있어야 해 이 화면에서는 못 만들었습니다.

내 계정 삭제

계획 · 할일 · 실행기록은 서버의 실제 데이터베이스에 저장되고, 로그인한 내 계정 소유의 행만 서버[Redis]가 돌려줍니다.

09_계정B_빈화면

13 / 31

계정 B로 가입 직후 — 계획 목록이 비어 있음(A의 계획이 안 보임) — 목록 조회에 상대 계정 행이 0건(검사 28) **PASS (검사 28)**

직접 조회 시도(id를 알아도) — RLS가 남의 행을 null로 돌려줌 — 서버가 거절함(검사 26)

B가 A의 계획 id(684b5034)를 `window.__db.plans.get()`으로 직접 조회 → 응답: null **PASS (검사 26)**

시도	결과
B가 A의 계획을 id로 직접 읽음 / A가 B의 계획을 읽음 (양방향)	둘 다 거절 — B→A읽기 거절(null) · A→B읽기 거절(null) — RLS(plans_select)가 양방향 모두 막음
B가 A의 계획을 지우려 함 / A가 B의 계획을 지우려 함 (양방향)	둘 다 반영 안 됨 — B가 시도한 A 계획 삭제·A가 시도한 B 계획 삭제 모두 반영되지 않음(deleted_at 그대로 null)
목록 조회에 상대 계정 행 섞임 (양방향)	0건 — B의 목록(1건)에 A의 계획 없음 · A의 목록(2건)에 B의 계획 없음
요청 본문에 남의 user_id를 적어 보냄(스푸핑)	트리거가 덮어씀 — 본문에 A의 user_id(299f1e60)를 적어 보냈지만 저장된 행은 실제 로그인한 B(ba91ddd5)로 찍힘

화면을 건너뛰고 서버가 무엇으로 거절하는지 (T07-C121, T07-C123, T07-C124)

위 표는 앱의 래퍼(`window.__db`)를 거친 결과라 "화면에 안 나온다"까지만 보입니다. 7.md는 그것을 막은 것으로 쳐 주지 않으므로, 아래 여덟 가지는 래퍼를 건너뛰고 REST 주소를 직접 두드려 **상태 코드와 응답 본문을 그대로** 받아 적었습니다.

보낸 요청	돌아온 응답
자격을 하나도 안 붙이고 <code>GET /rest/v1/plans</code>	401 No API key found in request PASS (검사 32)
공개 키만 붙이고 로그인 없이 <code>GET /rest/v1/plans</code>	200 이지만 0건 — 문은 열리지만 내 것이 하나도 없습니다
로그인 없이 <code>POST /rest/v1/plans</code>	401 42501 new row violates row-level security policy — 주인을 찍을 <code>auth.uid()</code> 가 없어 행 자체가 만들어지지 않습니다 PASS (검사 32)
남의 계획 한 건 을 집어 요청 (Accept: application/vnd.pgrst.object+json)	406 PGRST116 Cannot coerce the result to a single JSON object — "권한이 없다"가 아니라 "그런 행이 없다" 로 답해 id의 존재 여부까지 감춥니다 PASS (검사 33)
내 행의 주인만 남으로 바꾸는 <code>PATCH</code>	403 42501 new row violates row-level security policy — 같은 행·같은 방식인데 <code>user_id</code> 한 칸이 달라지자 서버가 되받습니다 PASS (검사 38)
주소에 남의 계정을 조건으로 적어 보냄 (<code>?user_id=eq.<A></code>)	200 이지만 0건 PASS (검사 35)
헤더에 남의 계정을 적어 보냄 (<code>x-user-id: <A></code>)	200 에 여러 건이지만 전부 로그인한 내 것 — 신원은 주소도 헤더도 아닌 JWT에서만 나옵니다 PASS (검사 35)
남의 계획 내용 을 고치는 <code>PATCH</code> (양방향)	200 이지만 0행 반영 · 거절 앞뒤로 값도 건수도 그대로 PASS (검사 34)

거절이 왜 403 하나로 통일되지 않는가. 이 서버는 남의 자료를 "막는" 것이 아니라 **아예 없는 것으로 만듭니다(RLS)**. 그래서 남의 행을 겨냥한 조회·수정은 "권한 없음"이 아니라 **그런 행이 없다**로 답합니다 — 한 건을 요구하면 406, 목록이나 수정이면 0건·0행입니다. 이 편이 403보다 덜 흘립니다: 403은 "그 id는 있는데 네 것이 아니다"까지 알려 주지만, 지금 응답

에서는 그 **id가 존재하는지조차 알 수 없습니다**. 반대로 내게 보이는 행을 건드리되 규칙을 어기는 요청 — 내 행의 주인을 남으로 넘기려는 PATCH — 에는 숨길 것이 없으므로 또렷하게 **403**이 돌아옵니다(검사 38).

거절 앞뒤로 반대편 자료가 그대로인지도 세었습니다 (T07-C122) — B→A 내용 수정 PATCH 200·0행 반영 · A→B 내용 수정 PATCH 200·0행 반영 (고치려 한 칸 `carried_from_review_id ← 66e5215c...`) · 거절 앞뒤로 값 그대로(A `null` · B `null`) · 건수도 그대로(A 3→3건 · B 2→2건) — 반대편에 새로 생긴 자료 없음

RLS 정책 정의는 `supabase/schema.sql` 의 `plans_select` · `plans_insert` · `plans_update` 등 각 표마다 반복되는 3줄(정책 5개 표 × 2~3개 정책)에 있습니다 — 표 하나당 소유자 조건이 딱 한 줄입니다.

⑤ AI와 나

구분	내용
AI에게 맡긴 일	Supabase Auth 연동(auth.js/AuthGate/AuthForm/AccountSection), user_id + stamp_owner 트리거 + RLS 스키마 재작성, 검사 18~40 설계·구현, capture.mjs/report.mjs를 인증 흐름에 맞게 재작성, session_id-auth.sessions 기반 즉시 세션 무효화(session_is_active()) 조사·구현, 계정 삭제 버튼을 로그아웃과 시각적으로 분리(위험 구역 스타일), 새 Supabase·Vercel 프로젝트 생성 과정에서 비밀번호가 필요 없는 모든 단계.
내가 직접 판단한 일	인증 방식으로 Supabase Auth(이메일+비밀번호)를 최종 확정, 새 프로젝트 이름(plandosee-auth)과 배포 이름(aleph-pds-auth) 확정, Supabase 새 프로젝트의 데이터베이스 비밀번호 입력과 'Confirm email' 끄기, 로컬/공개 주소에서 실제 계정으로 로그인해 데이터 이전을 직접 확인, 검사 23이 실패로 남은 것을 보고 '아직 못 막은 것'으로 넘기지 않고 실제로 막는 방법을 요구.
AI 제안을 따르지 않은 일(없다면 왜 없었는지)	없음 — 제시된 인증 방식·스키마·트리거 설계를 검토 후 그대로 채택했습니다. 다만 검사 19·20을 처음 돌렸을 때 오류 문구가 "undefined"로 나오는 버그(Error.message가 CDP 직렬화 경계에서 사라짐)를 발견해, 검사 스크립트 쪽의 직렬화 로직만 고쳤습니다 — 화면 코드는 그대로 두었습니다.

⑥ 아직 못 막은 것

아직 못 막은 것	왜 위험한가
무차별 대입(brute-force) 방지 없음	같은 계정에 비밀번호를 계속 시도해도 막는 장치가 없어 시간을 들이면 뚫릴 수 있습니다.
비밀번호 재설정 이메일 흐름 미구현	비밀번호를 잊으면 계정을 복구할 방법이 없습니다(시간이 부족해 다음으로 미룸).
2단계 인증 없음	비밀번호 하나만 뚫리면 끝입니다.
로그인 시도 로그 없음	누가 언제 실패했는지 남지 않아 이상 징후를 못 봅니다.
비밀번호 변경 기능 없음	토큰이 새어 나갔다고 의심될 때 비밀번호를 바꿔 이전 토큰을 한꺼번에 끊는 길이 없습니다. 지금 그 끊는 수단은 로그아웃뿐이고(카드 3), 로그아웃은 그 브라우저에서 직접 눌러야 합니다. 소스에 <code>updateUser</code> 를 부르는 자리가 아예 없습니다.

로그아웃 후 액세스 토큰 재사용 문제는 카드 3(`session_is_active()`)에서 실제로 막았습니다 — 이 표에는 남기지 않습니다.

카드 5 — 설명서로 묶고, 5일 써 보기

내 자료 전체를 파일 하나로 (T07-C133)

자료 화면의 "전체 내보내기" 버튼은 계획·개정 이력·할일·실행기록·고칠 점 다섯 표를 `exportedAt` 이 붙은 객체 하나로 묶어 `plandosee-auth-내보내기-YYYY-MM-DD.json` 파일로 내려받습니다(`src/components/ExportSection/ExportSection.jsx`, `src/api/exportAll.js`). 검사가 이 버튼을 실제로 눌러 안내 문구를 읽고, 받은 자료에 상대 계정의 행이 섞이지 않았는지까지 확인합니다.

PASS (검사 36) 판정 근거: "전체 내보내기"를 눌러 안내 "계획 2건 · 할일 0건 · 실행기록 0건을 내보냈습니다." · 표 5개(`plans`·`planRevisions`·`tasks`·`executionRecords`·`reviewNotes`) 총 3건이 `exportedAt`을 가진 객체 하나로 묶여 나옴 · 그 안에 상대 계정(A)의 행 0건

계정 삭제 (T07-C134)

The screenshot shows a user interface for account management. At the top, it says '내 계정' (My Account) and 'pds-auth-shot-b-1788852367767@example.com'. Below this is a '로그아웃' (Logout) button. A red warning box contains the text: '△ 위험 구역' (Danger Zone), '계정 삭제는 내 계획·할일·실행기록·고칠 점을 전부 지웁니다. 다만 가입 정보(이메일) 자체는 이 버튼으로 지워지지 않고 별도 절차가 필요합니다 — Auth 계정 레코드의 하드 삭제는 관리자 권한이 있어야 해 이 화면에서는 못 만들었습니다.' (Deleting an account deletes all plans, tasks, execution records, and points to fix. However, registration information (email) itself is not deleted by this button and requires a separate procedure — Hard deletion of Auth account records requires administrator privileges and cannot be done on this screen). At the bottom of the warning box is a red button labeled '내 계정 삭제' (Delete My Account).

11_계정관리 — 로그아웃·계정 삭제 버튼과 삭제 범위 안내 — 계정 삭제는 내 데이터 행까지만 지우고 가입 정보는 별도 절차를 화면에 명시(T07-C134)

계정 삭제는 로그인한 본인 권한으로 내 소유 행(계획·이력·할일·실행기록·고칠점)을 전부 하드 삭제하는 `delete_my_data()` 함수로 처리합니다. `auth.users` 의 가입 정보 자체는 `service_role` 관리자 API가 필요해 이 화면에서는 지우지 못했습니다 — 그 사실을 화면에 그대로 밝혀 T07-C134를 만족합니다.

1일차에 고정한 것 (T07-C04, T07-C05, T07-C06, T07-C08)

항목	고정한 값
답하려는 질문 한 문장 (T07-C04)	계획한 시간과 실제로 쓴 시간이 얼마나 벌어지는가?
관찰 지표 한 개 (T07-C05)	그날 실행기록의 실제 걸린 시간 합계
지표의 단위 (T07-C06)	분(minute)
계산 규칙	실행기록의 <code>startedAt</code> 을 Asia/Seoul 기준 날짜로 묶고, 그 날짜에 속한 <code>actualMinutes</code> 를 모두 더한다. 5일 전체에 같은 규칙을 적용한다 (T07-C08).

값이 이상할 때 어떻게 하는가 (T07-C23~C27)

경우	처리 방법
값이 빠졌을 때 (T07-C23)	그날을 0분으로 채우지 않고 빈 날로 둔다 . 없는 기록을 0으로 세면 '쉬는 날'과 '적지 못한 날'이 구분되지 않기 때문이다. 5일은 기록이 있는 서로 다른 날짜로만 센다. 실제로 이번 5일 중 9월 1일·3일에는 기록이 없고, 그 날들은 5일에 포함하지 않았다.
값이 중복될 때 (T07-C24)	같은 날짜에 실행기록이 여러 건이면 중복이 아니라 그날의 여러 작업으로 보고 모두 더한다 . 실제로 8월 31일은 5건, 9월 2일은 3건이 더해졌다. 같은 <code>id</code> 가 두 번 들어오는 경우만 중복이며, <code>upsert(onConflict: "id")</code> 가 덮어쓰므로 행이 늘지 않는다.
값이 유난히 될 때 (T07-C25)	버리지 않고 그대로 둔다 . 이번 5일에서 8월 31일 1370분은 다른 날의 30~115분과 크게 벌어지지만, 실제로 그날 그만큼 했으므로 지우면 기록이 아니게 된다. 대신 평균만 보지 않고 날짜별 값을 함께 싣는다 — 아래 표가 그것이다.
반올림 (T07-C26)	합계에는 반올림이 없다. 실제 걸린 시간을 분 단위 정수로 직접 입력 받고 나누는 연산이 없기 때문이다. 평균에만 소수 첫째 자리까지 남기고 그 아래를 반올림한다.
주 시작 요일 (T07-C27)	월요일 (ISO-8601). 이번 5일은 주 단위로 묶지 않았지만, 묶는다면 이 기준을 쓴다.

실제 5일 기록 (T07-C07, T07-C132)

Asia/Seoul 기준 **서로 다른 실제 날짜 정확히 5일**입니다. 아래 값은 내보내기 파일의 실행기록을 손으로 더한 것이고, `tools/check.mjs`의 검사 31이 같은 계산을 다시 해 대조합니다.

날짜 (KST)	그날 실행기록	실제 시간 합계
2026-08-31	5건	1370분
2026-09-02	3건	115분
2026-09-04	1건	30분
2026-09-05	1건	30분
2026-09-06	1건	30분
합계	11건	1575분
평균	—	315.0분

같은 값을 화면도 직접 보여 줍니다 (T07-C132)

자료 화면의 "**날짜별 실제 시간**" 표가 위와 같은 규칙으로 내 실행기록 전체를 날짜로 묶어 날짜별 값·합계·하루 평균을 그립니다(`src/components/DailySection/DailySection.jsx`, 규칙은 `src/core/dailyTotals.js`). 계획 하나 안을 보는 돌아보기와 달리 계획·할일 경계를 넘어 봅니다 — 카드 5가 묻는 것이 "며칠에 걸쳐 얼마나 했는가"이기 때문입니다.

날짜별 실제 시간

그날 실행기록의 실제 걸린 시간 합계 · 단위 분 · Asia/Seoul 기준

날짜 (KST)	그날 실행기록	실제 시간 합계
2026-09-01	1건	45분
2026-09-02	1건	30분
2026-09-03	1건	120분
2026-09-04	1건	25분
2026-09-05	1건	50분
합계	5건	270분
하루 평균	5일	54.0분

기록이 있는 서로 다른 날짜 5일의 합계 4시간 30분, 하루 평균 54.0분입니다. 기록이 없는 날은 0분으로 채우지 않고 세지 않으므로, 평균은 기록이 있는 날 수로 나눈 값입니다.

14. 날짜별_합계평균 — 화면이 그린 값 — 기록이 있는 날 5일 · 합계 270분 · 하루 평균 54.0분 — 화면이 날짜별 값과 합계·하루 평균을 직접 보여 줌 — 손으로 더한 값과 같은지는 검사 40이 대조(T07-C132)

위 화면은 **증빙용 스크래치 계정**에 서로 다른 날짜 5일을 심어 찍은 것입니다. 심사하는 분이 제 계정으로 로그인하지 않으므로(7.md 증거 보관 기준), 같은 표가 무엇을 그리는지를 새 계정에서 재현해 보였습니다. 제 실제 5일 값은 바로 위 표에 있습니다. 화면이 그린 숫자가 손으로 더한 값과 같은지는 검사 40이 매번 대조합니다 — **화면이 그린 날짜별 값** 2026-08-26 25분 · 2026-08-27 40분 · 2026-08-28 15분 · 2026-08-29 60분 · 2026-08-30 90분 · 화면 합계 230분 · 화면 평균 46분 — 같은 자료를 검사가 따로 더한 값(합계 230분 · 평균 46분 · 5일)과 전부 일치 **PASS (검사 40)**

계획 규칙과 그 변경 (T07-C09~C15)

1일차에 정한 계획 규칙 — "자격증 공부는 한 번 잡으면 끝까지 길게 붙잡고, 그 시간에는 그것만 한다."

바꾼 규칙 — "한 번에 길게 붙잡지 않는다. 하루 한 덩어리를 30분 안팎으로 줄이고, 다른 일과 병행하며 쉬엄쉬엄 한다."

항목	내용
바꾼 시각 (T07-C10)	2026-09-03 — 2일차 기록(2026-09-02) 뒤이고 3일차 기록(2026-09-04) 앞입니다 (T07-C09, T07-C12).
바꾼 이유 (T07-C11)	1·2일차처럼 한 번에 길게, 그리고 한 가지만 붙잡으니 중간에 손을 놓게 되었습니다 . 실제로 1일차(8/31)와 2일차(9/2) 사이 9월 1일에는 기록이 아예 없습니다. 시간을 줄이더라도 매일 이어지는 쪽이 낫다고 판단했습니다.
무엇을 가리키는가 (T07-C12)	1일차 = 2026-08-31(1370분), 2일차 = 2026-09-02(115분). 이 두 날의 방식을 바꾼 것입니다.

바꾸기 전과 뒤 — 같은 지표(T07-C13)·같은 단위(T07-C14)·같은 계산 규칙(T07-C15)으로

지표는 "그날 실행기록 실제시간 합계", 단위는 분, 계산 규칙은 KST 날짜로 묶어 `actualMinutes` 를 더하는 것. 양쪽에 똑같이 적용했습니다.

	날짜	날짜별 값	합계	하루 평균	기록이 있는 날 / 기간
바꾸기 전 (1·2일차)	08-31, 09-02	1370분, 115분	1485분	742.5분	2일 / 3일(9월 1일 비어 있음)
바꾼 뒤 (3·4·5일차)	09-04, 09-05, 09-06	30분, 30분, 30분	90분	30.0분	3일 / 3일(연속)

무엇이 달라졌나 — 하루 평균이 742.5분에서 30.0분으로 **712.5분 줄었습니다**. 대신 기록이 끊기지 않았습니다. 바꾸기 전에는 3일 중 2일만 남았고 그 사이 하루가 비었는데, 바꾼 뒤에는 **3일 연속**으로 남았습니다. 노린 것이 그것이었고 — 총량을 내주고 지속을 얻는 거래 — 지표가 그대로 보여 줍니다.

과제 6에서 이어 붙였다는 근거 (T07-C77-C78)

과제 6 최종 결과물은 aleph-pds.vercel.app이고 별개 프로젝트로 그대로 두었습니다. 소스 이력에서도 이어집니다 — 과제 6의 최종 커밋 `1227f3a` ("과제6 — 완료된 할일 시각 구분")가 과제 7 소스의 **조상**입니다. `git merge-base --is-ancestor 1227f3a HEAD` 로 확인했습니다. 과제 7의 첫 커밋은 `9eff9c5` 입니다.

검사 40개

`node tools/check.mjs --json` 실행 결과. 사람 눈이 아니라 이 명령 하나가 판정합니다. 1~17은 과제 6과 같은 계획·할일·실행기록·돌아보기 검사이고, 18~40이 이번 과제(인증·소유권)에서 새로 추가되었습니다. 그중 32~38은 앱의 래퍼를 건너뛰고 REST 주소를 직접 두드려 상태 코드와 응답 본문을 그대로 받아 적는 검사입니다.

#	카드	검사	결과	판정 근거
1	카드 1	계획에 기간·우선순위·성공기준·예상시간이 저장된다	PASS	계획 e07b86b7 — 기간·우선순위(high)·성공기준·예상시간(300분) 저장 확인
2	카드 1	계획을 고치면 새 개정본이 쌓이고 이전 개정본은 그대로 남는다	PASS	개정 1판 → 2판 · 1판은 여전히 300분(처음 그대로) · 최신판 480분
3	카드 2	할일에 마감일·우선순위·태그·예상시간을 저장하며 만들 수 있다	PASS	할일 ff585d68 — 마감일·우선순위·태그 2개·예상시간(90분) 저장 확인
4	카드 2	할일 내용을 고칠 수 있다	PASS	예상 시간 90 → 120분으로 수정됨
5	카드 2	완료로 바꾸고 다시 진행중으로 되돌릴 수 있다	PASS	todo → done(완료시각 있음) → todo(완료시각 비워짐)
6	카드 2	할일을 지우면 목록에서 사라지고 DB에는 소프트삭제로 남는다	PASS	목록에서 사라짐 · DB에는 deleted_at 채워진 채 남아 있음(하드 삭제 아님)
7	카드 2	검색·필터·정렬이 화면에 밝힌 기준대로 동작한다	PASS	검색 1건 · 필터(low) 1건 · 우선순위 오름차순 [low,medium,high,high]

#	카드	검사	결과	판정 근거
8	카드 3	실행기록에 시작·끝·실제 시간·막힘이 유가 저장되고 계획 값은 그대로다	PASS	실행기록 저장(시작·끝·90분·막힘이유) · 계획·개정 이력 값 변화 없음
9	카드 3	완료를 동시에 두 번 요청해도 완료 기록·집계가 한 번만 늘어난다	PASS	동시에 두 번 호출해도 완료시각이 같음(2026-09-08T07:25:07.04+00:00) — 실제로 쓰기가 일어난 건 한 번뿐
10	카드 4	돌아보기 집계(완료·지연·막힘·예상·실제·차이)가 정확하고 서버 계산과 일치한다	PASS	화면 계산 == 서버 RPC 계산 (전체 4·완료 0·지연 1·막힘 1·예상 180·실제 90·차이 -90)
11	카드 4	집계 숫자를 누르면 그 숫자가 나온 목록으로 간다	PASS	지연 숫자(1)와 드릴다운 대상 목록이 정확히 일치
12	카드 4	돌아보기의 고칠 점이 다음 계획으로 이어진다	PASS	다음 계획이 고칠 점(89d937fb)을 가리킴
13	카드 5	새로고침 뒤에도 값이 그대로 복원된다	PASS	새로고침 후에도 로그인 세션·id·값이 동일
14	카드 5	스크립트 모양 글자를 저장해도 실행되지 않고 글자 그대로 보인다	PASS	저장한 스크립트 모양 글자가 alert 없이 화면에 문자 그대로 보임
15	카드 5	비로그인 첫 화면에 로그인 필수 안내 문구가 있다	PASS	AuthForm의 안내 문구가 소스와 정확히 일치

#	카드	검사	결과	판정 근거
16	카드 5	빌드 산출물 어디에도 service_role 비밀키가 없다	PASS	빌드 산출물 1개 파일 검사 — service_role 문자열 0건
17	부가	계획을 지우면 목록에서 사라지고 DB에는 소프트 삭제로 남는다	PASS	목록에서 사라짐 · DB에는 deleted_at 채워진 채 남아 있음(하드 삭제 아님)
18	카드 1	가입 → 로그아웃 → 로그인 순서대로 세션 상태를 바꾼다	PASS	가입 직후 세션 있음 → 로그아웃 후 세션 null → 재로그인 세션 있음 (user 299f1e60)
19	카드 1	같은 이메일로 두 번 가입하면 거절된다	PASS	같은 이메일 재가입 거절: "User already registered"
20	카드 1	존재하지 않는 계정과 비밀번호만 틀린 계정의 오류 문구가 같다	PASS	두 실패 응답의 오류 문구가 동일: "Invalid login credentials"
21	카드 1	로그인하지 않으면 자료 화면 자체가 DOM에 없다	PASS	로그인 폼만 있고, 계획 목록·내보내기 등 자료 화면 요소는 DOM에 아예 없음
22	카드 2	같은 비밀번호로 만든 두 계정이 서로 다른 사용자로 분리된다	PASS	같은 비밀번호로 만든 두 계정의 id가 다름 (A 299f1e60 / B ba91ddd5) — 실제 해시값 (bcrypt, salt 다름)은 Supabase SQL 편집기로 auth.users를 조회해 설명서에 수기로 첨부합니다
23	카드 3	로그인 상태 조회는 성공, 로그아웃 뒤 같은 토큰을 재사용해도 내 자료는 하나도 안 보인다	PASS	로그인 상태 조회 200(내 계획 2건) → 로그아웃 뒤 같은 토큰 재사용 200이지만 0건 — session_is_active()가 auth.sessions에서 지워진 세션을 찾지 못해 행을 전부 걸러냄(토큰이 서명상 유효해도 자료는 하나도 못 봄)

#	카드	검사	결과	판정 근거
24	카드 3	액세스 토큰이 화면 URL 어디에도 실리지 않는다	PASS	현재 주소에 access_token/refresh_token 문자열 없음: https://aleph-pds-auth.vercel.app/
25	카드 3	액세스 토큰의 만료 시간이 발급 후 약 1시간이다	PASS	exp - iat = 3600초 (약 1시간)
26	카드 4	계정 A·B가 서로의 계획을 id로 직접 읽으면 거절된다(양방향)	PASS	B→A읽기 거절(null) · A→B읽기 거절(null) — RLS(plans_select)가 양방향 모두 막음
27	카드 4	계정 A·B가 서로의 계획을 고치거나 지우려 하면 거절된다(양방향)	PASS	B가 시도한 A 계획 삭제·A가 시도한 B 계획 삭제 모두 반영되지 않음(deleted_at 그대로 null)
28	카드 4	목록 조회 응답에 상대 계정의 행이 0 건이다(양방향)	PASS	B의 목록(1건)에 A의 계획 없음 · A의 목록(2건)에 B의 계획 없음
29	카드 4	요청에 남의 user_id를 적어 보내도 트리거가 내 id로 덮어쓴다	PASS	본문에 A의 user_id(299f1e60)를 적어 보냈지만 저장된 행은 실제 로그인한 B(ba91ddd5)로 찍힘
30	카드 5	빌드 산출물에 새 명명(secret key)의 비밀값도 없다	PASS	빌드 산출물 1개 파일 검사 — 실제 secret key 값·SERVICE_ROLE 대입 0건 (접두어 판별 코드는 라이브러리 코드라 제외)
31	카드 5	화면의 5일 합계·평균이 손 계산과 같다	PASS	plandosee-auth-내보내기-2026-09-08.json 기준 서로 다른 실제 날짜 정확히 5일 — 2026-08-31 1370분 · 2026-09-02 115분 · 2026-09-04 30분 · 2026-09-05 30분 · 2026-09-06 30분 · 합계 1575분 · 평균 315.0분 (지표: 그날 실행기록 실제시간 합계, 단위 분, KST)
32	카드 4	로그인하지 않은 채 자료를 직접 요청하면 거절된다(읽기·쓰기)	PASS	자격 없이 GET /rest/v1/plans → 401 "No API key found in request" · 공개키만 붙이고 로그인 없이 GET → 200이지만 0건(RLS가 전부 걸러냄) · 로그인 없이 POST /rest/v1/plans → 401 (42501) "new row violates row-level security policy for table W"plansW"" — 주인을 찍을 auth.uid()가 없어 행 자체가 만들어지지 않습니다

#	카드	검사	결과	판정 근거
33	카드 4	남의 자료 한 건을 집어 요청하면 존재 자체를 감추는 응답이 돌아온다	PASS	같은 주소·같은 방식으로 내 계획 한 건 요청 → 200 · 남의 계획(A) 한 건 요청 → 406 "PGRST116" "Cannot coerce the result to a single JSON object" — "권한 없음"이 아니라 "그런 행이 없다"로 답해 id의 존재 여부까지 감춥니다
34	카드 4	남의 계획 내용을 고치려는 요청이 양 방향으로 거절되고 건수도 그대로다	PASS	B→A 내용 수정 PATCH 200·0행 반영 · A→B 내용 수정 PATCH 200·0행 반영 (고치려 한 칸 carried_from_review_id ← 66e5215c...) · 거절 앞뒤로 값 그대로(A null · B null) · 건수도 그대로(A 3→3건 · B 2→2건) — 반대편에 새로 생긴 자료 없음
35	카드 4	주소와 요청 헤더에 남의 계정을 적어 보내도 내 자료만 돌아온다	PASS	주소에 A의 user_id를 조건으로 적어 보냄(?user_id=eq.299f1e60...) → 200이지만 0건 · 헤더에 x-user-id: A를 적어 보냄 → 200에 2건이지만 전부 로그인한 B(ba91ddd5)의 것 — 신원은 주소도 헤더도 아닌 JWT에서만 나옵니다
36	카드 5	내 자료 전체를 파일 하나로 내보낼 수 있고 남의 자료가 섞이지 않는다	PASS	"전체 내보내기"를 눌러 안내 "계획 2건 · 할일 0건 · 실행기록 0건을 내보냈습니다." · 포 5개(plans·planRevisions·tasks·executionRecords·reviewNotes) 총 3건이 exportedAt을 가진 객체 하나로 묶여 나옴 · 그 안에 상대 계정(A)의 행 0건
37	카드 2	로그인 요청과 응답 어디에도 비밀번호 원문이 남지 않는다	PASS	POST /auth/v1/token?grant_type=password → 200 · 응답 본문에 비밀번호 원문 0건 (돌아온 것은 access_token·token_type·expires_in·expires_at·refresh_token·user·weak_password) · localStorage에 저장된 세션·화면 글자·주소창까지 네 곳 모두 원문 0건
38	카드 4	내 행의 주인을 남에게 넘기려 하면 403으로 거절된다	PASS	같은 주소·같은 방식 PATCH — 내 행을 내 것으로 두고 고치면 200, 주인만 A로 바꾸려 하면 403 (42501) "new row violates row-level security policy for table W"plansW"" · 다시 읽어 보면 주인은 여전히 B(ba91ddd5) — 달라진 것은 user_id 한 칸뿐인데 서버가 403으로 되받습니다
39	카드 1	과제 6에 넣어 둔 내 자료가 과제 7 계정으로 그대로 옮겨져 있다	PASS	과제 6 내보내기(plandosee-내보내기-2026-09-02-과제7이전용.json)의 27건이 과제 7 계정 내보내기(plandosee-auth-내보내기-2026-09-08.json)에 id 그대로 전부 있음 — plans 4건 중 4건 · planRevisions 5건 중 5건 · tasks 7건 중 7건 · executionRecords 5건 중 5건 · reviewNotes 6건 중 6건 · 빠진 행 0건 (id가 같다는 것은 다시 입력한 것이 아니라 실제로 옮겨졌다는 뜻입니다)
40	카드 5	화면이 그린 날짜별 합계·평균이 손으로 더한 값과 같다	PASS	화면이 그린 날짜별 값 2026-08-26 25분 · 2026-08-27 40분 · 2026-08-28 15분 · 2026-08-29 60분 · 2026-08-30 90분 · 화면 합계 230분 · 화면 평균 46분 — 같은 자료를 검사가 따로 더한 값(합계 230분 · 평균 46분 · 5일)과 전부 일치

개인정보와 비밀값

검사 대상	결과
증빙 촬영 중 나간 외부(비-Supabase) 요청	fonts.googleapis.com · fonts.gstatic.com
service_role 비밀키가 빌드 산출물에 포함	PASS (검사 16)
secret key(신 명명):SERVICE_ROLE 문자열이 빌드 산출물에 포함	PASS (검사 30)
JWT 서명 비밀키가 소스·번들에 포함	없음 — Supabase 프로젝트 설정(대시보드)에만 있고 클라이언트 코드에는 없음
스크립트 모양 글자 실행 여부	PASS (검사 14)

publishable(구 anon) 키는 Supabase 설계상 브라우저에 노출되는 것이 정상입니다 — 실제 접근 통제는 이 키가 아니라 RLS가 합니다. 노출되면 안 되는 것은 secret(구 service_role) 키뿐이고, 이 키는 스키마를 설정할 때 Supabase SQL 편집기에서만 쓰고 저장소·소스·배포 환경 어디에도 넣지 않습니다.

플랜두씨 다이어리 2

계획(Plan) → 실제로 한 일(Do) → 돌아보기(See) · 기준
시간대 Asia/Seoul (UTC+9)

오늘 2026-09-08

pds-auth-shot-b-1788852367767@example.com

I 계획

계획은 고쳐도 지워지지 않고, 새 개정본으로 쌓입니다

아직 계획이 없습니다. 아래에서 하나 만듭니다.

제목	기간	우선순위	예상 시간	개정
----	----	------	-------	----

I 새 계획

기준 시간대 Asia/Seoul (UTC+9)

제목

예: ALEPH 과제 6 진행

시작일

연도-월-일



종료일

연도-월-일



우선순위

보통



예상 시간 (분)

600

성공 기준

무엇이 되면 이 계획을 완료로 볼지 한두 문장으로

메모 (선택)

계획 만들기

할 일

먼저 위에서 계획을 하나 선택하거나 만듭니다.

실행 기록

할 일 목록에서 실행기록을 선택하세요

위 할 일 목록에서 “실행기록”을 눌러 어느 할 일에 적을지 고릅니다.

I 돌아보기

먼저 계획을 하나 선택합니다.

I 날짜별 실제 시간

그날 실행기록의 실제 걸린 시간 합계 · 단위 분 ·
Asia/Seoul 기준

아직 실행기록이 없습니다. 할 일을 고르고 실행기록을
남기면 여기에 날짜별로 쌓입니다.

I 내 자료 내보내기

계획 · 이력 · 할일 · 실행기록 · 고칠 점 전체를 파일 하나로

전체 내보내기

I 내 계정

pds-auth-shot-b-1788852367767@example.com

로그아웃

⚠ 위험 구역

계정 삭제는 내 계획·할일·실행기록·고칠 점을 전부 지웁니다. 다만 **가입 정보(이메일)** 자체는 이 **버튼으로 지워지지 않고 별도 절차가 필요합니다**

— Auth 계정 레코드의 하드 삭제는 관리자 권한이 있어야 해 이 화면에서는 못 만들었습니다.

내 계정 삭제

계획·할일·실행기록은 서버의 실제 데이터베이스에 저장되고, 로그인한 내 계정 소유의 행만 서버(RLS)가 돌려줍니다.

13_모바일_375_로그인화면 — 가로 스크롤 없음 (본문 폭 375px)